

MANUAL TÉCNICO

Arreglos y matrices:

- **tablero(String[][]):** Matriz estática bidimensional nativa de 10x10. Aunque el área funcional del estacionamiento requerida era de 8x8, se expandió a 10x10 para simular la vías exteriores.

Variables globales:

- **tarifa(double):** Constante del tipo decimal de valor 10.00 que se utiliza para calcular el valor unitario del uso del estacionamiento.
- **vehiculos_ingresados(int):** Variable de tipo entero creada para llevar el cálculo de cuántos vehículos se encuentran dentro del estacionamiento. se incrementa cuando ingresan vehículos y disminuye cuando salen.
- **vehiculos_cobrados(int):** Variable de tipo entero creada específicamente para llevar el control de cuántos vehículos ingresaron durante toda la ejecución del programa llevada de la mano para hacer el cálculo de los ingresos totales del estacionamiento, esta variable incrementa cuando ingresan vehículos y no disminuye cuando salen..
- **total_recaudado(double):** Variable de tipo double creada para llevar el cálculo financiero de todos los vehículos ingresados en el estacionamiento durante la ejecución del programa.
- **(entrada_fila, entrada_columna, salida_fila, salida_columna):** Variables de tipo entero que ayudan a llevar los índices exactos en el perímetro para posiciones de forma correcta la entrada y la salida del estacionamiento.

Métodos:

- **main(String[] args):** En este módulo se encuentra el menú principal y los llamados a los demás métodos que componen el programa.
- **inicializarTablero():** Algoritmo de inicialización, ejecuta un barrido por las 100 celdas de la matriz del estacionamiento, aplica la lógica condicional para identificar los límites y rellenar los bordes don '=' y 'L' para el interior del estacionamiento identificando un espacio vacío.
- **generarEntradaSalida():** Su función es determinar matemáticamente las posiciones válidas para crear la entrada 'E' y la salida 'S' de forma aleatoria, así como también evita que estas terminen posicionándose en el mismo lugar.
- **ingresarVehiculo():** La función de este módulo es ingresar los vehículos solicitando al usuario la entrada de la placa, pasar los respectivos filtros para que dicha placa sea aceptada por el programa, hacer el respectivo cobro como ingresar el vehículo al estacionamiento.

- **retirarVehiculo():** Le solicita al usuario la placa del vehículo que quiere retirar también se le aplican los mismos filtros que al módulo anterior para verificar que sea válida, identifica la posición (fila y columna) del vehículo para luego pedir confirmación de que ese vehículo sea retirado del estacionamiento.
- **mostrarEstacionamiento():** Su función es mostrar el estacionamiento completo, sus lugares libres también muestra los espacios vacíos como los espacios ocupados.
- **buscarVehiculo():** Solicita una placa y muestra su posición (fila y columna) en la que se encuentra.
- **mostrarRuta():** Calcula la ruta más corta entre las vías exteriores de la entrada y salida del estacionamiento generada aleatoriamente.
- **ingresosEstacionamiento():** Su función es llevar el cálculo de los ingresos totales generados durante toda la ejecución del programa así como también el conteo de todos los vehículos ingresados.

Validaciones:

- **Capacidad máxima:** Usando `if(vehiculos_ingresados == 64)` ayuda a evitar que se ingrese un vehículo cuando el estacionamiento ya tiene todos sus espacios ocupados.
- **Bloqueo de minúsculas:** Con `if(!placa.equals(placa.toUpperCase()))` evalúa que la placa ingresada este en minúsculas, actúa de forma que agarra la placa ingresada, vuelve todas las letras en mayúscula, luego las compara y si son iguales pasa el filtro, de lo contrario significa que alguna letra esta en minúscula y salta un mensaje de error.
- **Verificación del patron:** Usando `if(!placa.matches("^P[0-9]{3}[A-Z]{3}$"))` obliga que la placa tenga exactamente 7 caracteres y deba tener el patrón P###LLL.
- **Duplicidad de placas:** `.equals` ayuda a saber si la placa ingresada exista ya previamente en el estacionamiento, asegura que no sea idéntica.
- **Prevención de colisiones:** `if(!tablero[fila][columna].equals("L"))` evita que un auto se estacione encima de otro revisando que la celda seleccionada sea igual a 'L' si la celda es distinta salta un error indicando que el espacio ya está ocupado.
- **Filtro de montos:** `if(monto<0)` bloque el proceso con montos negativos e `if(monto<tarifa)` le advierte al usuario que el monto es insuficiente para poder ingresar el vehículo.

Decisiones de implementación:

- **Máscara visual:** En la matriz (tablero) que guarda la información original (placa) se le agregó un filtro que agarra la placa y la proyecta con 'A' que ayuda a visualmente identificar que el espacio contiene un vehículo.
- **Detectores de posición:** Para localizar las placas en los módulos 2 y 4 se inicializó una variable "temporal" en -1 para que si la fila y columna existiesen cambiarán de valor, identificando que dicha posición existe, en caso de que uno de los dos falle da a entender que la posición no existe y por ende la placa no esté ingresada en el estacionamiento.
- **Generación de entrada y salida:** Para evitar la posibilidad de que la entrada y/o salida se generaran en las esquinas se dividió en 2 tareas, la generación del lado (arriba, abajo, izquierda y derecha) para la selección del borde en la cual estará la entrada o salida y la generación de la posición (del 1 al 8) para seleccionar la posición de la celda, esto hace que no haya posibilidad de que se generen en las esquinas.
- **Duplicidad en entrada y salida:** con el do-while se asegura que bajo ninguna circunstancia la entrada y salida se generen en el mismo punto.